

Git & GitHub Cheatsheet

Practical Commands and Workflows

Git Basics

Initialize Repository

```
git init
```

Create a new local Git repository.

Check Status

```
git status
```

See:

- modified files
- staged files
- untracked files

Add Files

```
git add .  
git add file.py
```

Move files into staging area.

Commit

```
git commit -m "message"
```

Save staged changes into Git history.

Branches

Create Branch

```
git switch -c feature
```

Create and switch to new branch.

Switch Branch

```
git switch main
```

Move between branches.

List Branches

```
git branch
```

Show local branches.

Delete Branch

```
git branch -d feature
```

Delete merged branch.

HEAD

HEAD Meaning

HEAD = your current location in Git.

Usually:

```
HEAD -> main -> latest commit
```

Detached HEAD:

```
git switch --detach COMMIT
```

Temporarily visit old commit.

Working With Old Commits

See Old Commit

```
git switch --detach HASH
```

Safely inspect old project state.

Compare Commits

```
git diff HASH
```

See differences with old commit.

Restore Old File

```
git restore --source=HASH file.py
```

Restore specific file from old commit.

Reset vs Revert

Reset

```
git reset --soft HEAD~1  
git reset --mixed HEAD~1  
git reset --hard HEAD~1
```

Move branch backward.

Soft: keep staged changes.

Mixed: keep unstaged changes.

Hard: delete changes.

Revert

```
git revert HASH
```

Create NEW commit that undoes old commit.

Important:

- Reset rewrites history
- Revert preserves history

Stash

Save Unfinished Work

```
git stash
```

Temporarily hide local changes.

Restore Stash

```
git stash pop
```

Restore and remove stash.

Apply Without Removing

```
git stash apply
```

Restore stash but keep backup.

See Stashes

```
git stash list
```

Show all saved stashes.

Delete Stash

```
git stash drop stash@{0}
```

Delete stash permanently.

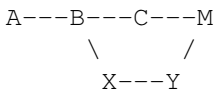
Merge vs Rebase

Merge

```
git merge main
```

Combine histories using merge commit.

Diagram:

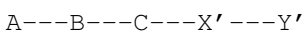


Rebase

```
git rebase main
```

Replay commits on newer base.

Diagram:



Important:

- Rebase rewrites history
- Avoid rebasing shared public commits
- Good for private feature branches

Fetch vs Pull

Fetch

```
git fetch
```

Download remote updates safely.
Does NOT modify current branch.

Pull

```
git pull
```

Equivalent to:

```
git fetch + git merge
```

Pull With Rebase

```
git pull --rebase
```

Replay local commits on latest remote commits.

GitHub Basics

Connect Remote

```
git remote add origin URL
```

Connect local repo to GitHub.

See Remotes

```
git remote -v
```

Show remote URLs.

Push

```
git push -u origin main
```

Upload branch to GitHub.

-u means:

Set upstream tracking branch.

Push Conflicts

Push Rejected

Possible error:

```
non-fast-forward
```

Meaning: remote changed before your push.

Solution:

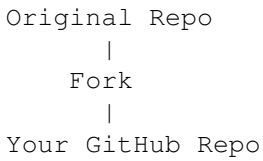
```
git pull --rebase
```

Resolve conflicts if needed.

Fork Workflow

Fork

Fork creates your own GitHub copy.



Clone Your Fork

```
git clone YOUR_FORK_URL
```

Add Upstream

```
git remote add upstream ORIGINAL_URL
```

origin = your fork

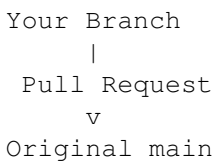
upstream = original repo

Pull Request Workflow

Typical Workflow

1. Fork repository
2. Clone your fork
3. Create branch
4. Make changes
5. Commit
6. Push branch
7. Open Pull Request

Diagram:



Push Branch

```
git push -u origin feature
```

Open Pull Request

On GitHub:

- Compare & Pull Request
- feature -> main

Useful Commands

```
git log --oneline
```

Compact history.

```
git diff
```

See modifications.

```
git reflog
```

Recover lost commits/history.

```
git rm file.py
```

Delete tracked file.

```
git restore file.py
```

Undo file modifications.

Important Professional Rules

- Use branches for features
- Avoid working directly on main
- Prefer rebase on your own branches
- Prefer merge/revert on shared public branches
- Use stash for temporary unfinished work
- Pull before push in team projects
- Write meaningful commit messages
- Small pull requests are easier to review

Mental Models

Commit = save snapshot

Branch = separate line of development

HEAD = where you currently are

Merge = combine histories

Rebase = rewrite commits on new base

Fork = your own repository copy

Pull Request = request to merge changes